



Common Problems

Movie Ticket Booking 🎬

By Evan King · Published Feb 3, 2026 ·  medium

Understanding the Problem

🎬 What is a Movie Ticket Booking System?

A movie ticket booking system (like Fandango) lets users search for movies, browse theaters and showtimes, select specific seats from a seat map, and reserve tickets. The system manages seat availability across multiple theaters, each with multiple screens, and prevents two people from booking the same seat.

Requirements

Your interview starts with this prompt:

"Design a movie ticket booking system similar to BookMyShow that allows users to browse movies, select theaters and showtimes, book tickets, and manage reservations."

That's broad. Before writing a single requirement, you need to narrow this down with targeted questions.

Clarifying Questions

Focus your questions on core operations, scope boundaries, and constraints.

You: "When you say 'browse movies,' is that full-text search, fuzzy matching, or just simple title lookup?"

Interviewer: "Simple text matching on the movie title. Nothing fancy."

That means we can iterate through our movie list, check if each title contains the search term, and return matches. No need for Elasticsearch, inverted indexes, or ranking algorithms. With a few hundred movies in memory, a linear scan takes microseconds.

You: "How does seat selection work? Does the user pick specific seats from a map, or does the system auto-assign? And can they book more than one seat at a time?"

Interviewer: "Users pick specific seats from a seat map. And yes, they can book multiple seats in one transaction."

So we're building a seat picker, not just a ticket counter. That means we need per-seat availability tracking. The actual UI for rendering the seat map is out of scope, but our system needs to expose which seats are available so a frontend could display them.

You: "Are we designing for a single theater or multiple? And do theaters have multiple screens?"

Interviewer: "Multiple theaters, each with multiple screens. A user can search for a movie and see where it's playing, or go to a specific theater and see what's on."

Multi-theater with multiple screens. Two entry points into the system: search by movie title globally, or browse a specific theater's offerings. Both paths funnel into picking a showtime, then picking seats. We need to support both directions efficiently.

You: "Do different screens have different seat configurations? Or can we standardize?"

Interviewer: "Standardize it. Every screen has the same layout: rows A through Z, seats 0 through 20."

Big simplification. One constant seat layout for every screen in the system means we don't need to model per-screen configurations.

You: "What does 'manage reservations' include? Cancel, reschedule, modify?"

Interviewer: "Cancel only. If someone wants a different showtime, they cancel and rebook."

No rescheduling logic. That keeps the reservation model simple.

You: "A couple of scoping questions: are there different seat types with different prices? And is payment processing in scope?"

Interviewer: "No to both. All seats are identical, and payment is out of scope. Assume it always succeeds."

Two fewer features to model. No pricing tiers, no payment state machine.

You: "What about concurrency? If two people try to book the same seat at the same time?"

Interviewer: "Handle it. Exactly one should succeed."

Concurrency is a core requirement.

Final Requirements

FINAL REQUIREMENTS



Requirements:

1. Users can search for movies by title
2. Users can browse movies playing at a given theater
3. Theaters have multiple screens; all screens share the same seat layout (rows A-
4. Users can view available seats for a showtime and select specific ones
5. Users can book multiple seats in a single reservation; booking returns a confir
6. Concurrent booking of the same seat: exactly one succeeds
7. Users can cancel a reservation by confirmation ID, releasing the seats

Out of Scope:

- Payment processing (assume payment always succeeds)
- Variable seat layouts or seat types (all seats identical)
- Rescheduling (cancel and rebook instead)
- UI / rendering

Core Entities and Relationships

Scanning the requirements gives us a pool of candidates: Theater, Movie, Screen, Seat, Showtime, Reservation, and something to orchestrate it all. Not every candidate will become its own class, but they're all worth listing out. Each one shows up somewhere in the design, whether as a class, a value object, a field, or a constant.

Theater - A named location that hosts showtimes. Users ask "what's playing at AMC tonight?" Theater is a first-class entity because it owns a schedule of showtimes and users query against it directly. It becomes the natural container for everything happening at a given location.

Movie - Users search for movies by title. The same movie plays across multiple theaters and multiple showtimes. Movie is what ties those showtimes together. When someone searches "Inception," we need to find every theater and time slot showing it. Movie is a first-class search target that connects results across the system.

Screen - A movie plays on a specific screen inside a theater. Instinct says "make it a class." But we just got the interviewer to agree that every screen uses the same seat layout. Without variable layouts, Screen doesn't track state, enforce rules, or own any data that differs from one screen to the next. It's just a label, like "Screen 3" telling the customer which room to walk into. So we can demote it to a string field on Showtime.



This is the kind of scoping question that pays for itself. *"Can we standardize seat layouts?"* eliminated an entire entity and all the indirection that comes with it.

Showtime - A specific screening of a movie at a particular time and screen. "Inception at 7pm on Screen 3 at AMC." Since each screening has its own set of seats, Showtime is where we track which seats are booked and handle the case where two users try to grab the same seat.

Reservation - When a booking succeeds on a Showtime, the user receives a confirmation ID. They'll use that ID to cancel later. The reservation holds the seat list so cancellation knows what to release.

Seat - Users pick seats, which suggests Seat might need its own class. But consider what a Seat actually does in this system. It doesn't change state. It doesn't enforce rules. It doesn't maintain behavior. It's just an identifier: "A5" means row A, seat 5. A string works fine. No need for a custom class when the language gives us strings with built-in equality and hashing.

BookingSystem - Who handles search? "Find all movies matching 'Inception'" cuts across all theaters and their showtimes. No single entity owns that question. Same with "what's playing at AMC tonight?" which requires walking a theater's showtimes and collecting distinct movies. We need a top-level entry point that holds theaters and provides these cross-cutting queries. This is our orchestrator.



A good heuristic for whether something deserves its own class is to ask whether it connects other entities, gets queried against, or needs its own behavior. Movie is a class because users search by it and it ties showtimes together across theaters. Screen is just a label telling the customer which room to walk into.

Final Entities

Entity	Responsibility	Type
BookingSystem	Orchestrator. Owns theaters. Entry point for search and cross-theater queries.	Class
Movie	Searchable entity. Title plus ID. Ties showtimes together across theaters.	Class
Theater	Named location. Owns a list of showtimes.	Class
Showtime	A specific screening. Tracks per-seat availability and handles booking concurrency.	Class
Reservation	User's booking reference. Stores confirmation ID and which seats were booked. Used for cancellation.	Class
Seat	A string identifying a specific seat, e.g. "A5" . No behavior, just an identifier.	String
Screen	Label identifying which room a showtime is in, e.g. "Screen 3".	String field

As for how these entities relate to one another:

KEY RELATIONSHIPS



```
BookingSystem → List<Theater>
Theater → List<Showtime>
Showtime → Theater (back-reference for navigation)
Showtime → Movie (reference)
Showtime → List<Reservation> (booking records for this showtime)
Reservation → Showtime (back-reference for cancellation routing)
Reservation → List<string> (e.g., ["A5", "A6"])
```

Class Design

We know what entities we're building. Now we need to define what each one stores and what operations it exposes. For every class, we'll ask two things: what state does this entity need to satisfy the requirements, and what does the outside world need to do with it?

Starting with the orchestrator, then working down to the most granular types.

BookingSystem

BookingSystem is the entry point for the system. Every user-facing operation flows through here, whether that's searching for a movie, browsing a theater's schedule, booking seats, or canceling a reservation. It manages the collection of theaters and coordinates the booking flow between the caller and the correct Showtime.

Requirement	What <code>BookingSystem</code> must track
"search for movies" / "browse movies at a theater"	Collection of all theaters (movies discoverable through showtimes)
"book seats" / "cancel reservation"	Need to route to the correct showtime

```
BOOKINGSYSTEM 
```

```
class BookingSystem:
    - theaters: List<Theater>

    + searchMovies(title: string) → List<Showtime>
    + getShowtimesAtTheater(theater: Theater) → List<Showtime>
    + book(showtimeId: string, seatIds: List<string>) → Reservation
    + cancelReservation(confirmationId: string)
```

`searchMovies` does case-insensitive substring matching against movie titles and returns all future showtimes for matching movies across all theaters. This gives users the actionable information they need: not just that "Inception" exists, but where and when they can actually book it.

`getShowtimesAtTheater` accepts a Theater object and returns all its future showtimes. The UI can group these by movie to show "what's playing tonight" with times for each film. This avoids the N+1 query pattern where you'd fetch movies first, then query showtimes for each one separately. Since this is an in-memory system, passing the Theater object directly is cleaner than using IDs.

These two methods cover both entry points into the system from searching globally by movie title, to browsing what's playing at a specific theater.

`book` creates a Reservation (generating a confirmation ID) and hands it to the correct Showtime for `atomic` validation and storage. `cancelReservation` locates the reservation, routes to its Showtime, and tells it to cancel. How BookingSystem finds reservations and showtimes efficiently is an implementation detail we'll tackle later.

There's no top-level `reservations` field because each Showtime owns its own reservations, co-located with the seat state they modify. Whether BookingSystem needs its own movie index is an implementation detail we'll address later.

Theater

Theater represents a physical location where movies are shown. From a user's perspective, they care about two things: what theater they're going to, and what's playing there. That makes Theater a natural container for showtimes.

Requirement	What <code>Theater</code> must track
"browse movies at a theater"	Theater identity (name, ID) and its list of showtimes
"Theaters have multiple screens"	Showtimes include a screen label

THEATER



```
class Theater:
    - id: string
    - name: string
    - showtimes: List<Showtime>

    + getShowtimes() → List<Showtime>
    + getShowtimesForMovie(movie: Movie) → List<Showtime>
```

`getShowtimesForMovie` is a convenience filter that iterates the theater's showtimes and returns those matching the given movie. Theater knows nothing about booking logic. It's a container for showtimes, not a participant in the booking flow.

Showtime

Showtime is where the interesting design decisions live. This is the actual bookable unit, like "Inception at 7pm on Screen 3 at AMC." Every booking and cancellation ultimately runs through a specific Showtime. Seat availability, reservation storage, and concurrency control all converge on this one entity.

Think about the behavior first. A user opening the seat picker needs to know which seats are free and which are taken. The booking flow needs to check availability, claim the seats, and handle the case where someone else grabbed them first. Cancellation needs to find the right reservation and release those seats. All of that behavior centers on one question: what is the current booking state of each seat for this showtime?

What does Showtime need to track to support that?

Requirement	What Showtime must track
"view available seats for a showtime"	Which seats are currently booked
"book multiple seats in a single reservation"	The reservation and its seats
"cancel a reservation, releasing the seats"	Which reservations exist, so we can find and remove them

At first glance the table seems to ask for two things: a way to track reservations, and a way to track which seats are booked. But look at what a reservation actually contains. It holds the list of seats that were booked. So if you have all the reservations for a showtime, you already know exactly which seats are taken, just by scanning each reservation's seat list. The reservations ARE the seat state. One list gives us both.

That means booking is just adding a reservation to the list (and its seats become taken), while cancellation is removing one (and its seats become free). Both operations modify the same single data structure, in one place, which means no cross-object consistency to worry about.

SHOWTIME



```
class Showtime:  
    - id: string
```



```
- theater: Theater
- datetime: DateTime
- screenLabel: string
- movie: Movie
- reservations: List<Reservation>

+ getId() → string
+ getTheater() → Theater
+ getDatetime() → DateTime
+ getMovie() → Movie
+ isAvailable(seatId: string) → boolean
+ getAvailableSeats() → List<string>
+ book(reservation: Reservation)
+ cancel(reservation: Reservation)
```



Notice that Showtime has a theater reference while Theater has a showtimes list. This bidirectional relationship exists so users can navigate from reservation → showtime → theater to see where their booking is. In practice, the caller creates the Theater first, then creates its Showtimes passing the Theater reference. The bidirectional link is established once during setup and never changes. Theater is the aggregate root and Showtime's theater reference is purely for navigation, not mutation.

The `reservations` list is the source of truth for this showtime's booking state. Every booked seat is represented exactly once, inside exactly one reservation. To check if a seat is taken, we scan the reservations. To get all available seats, we collect what's booked and subtract them from the constant seat layout (rows A-Z, seats 0-20). No separate data structure needed.

`isAvailable` checks whether a seat appears in any existing reservation's seat list.

`getAvailableSeats` generates every seat from the constant layout and filters out the booked ones, giving the UI what it needs to render the seat picker.

`book` takes a Reservation object (already created by BookingSystem with a confirmation ID and seat list), validates that all requested seats are still available, and stores the reservation atomically. If any seat is taken, the booking fails with no state change. `cancel` removes a Reservation from the list, and the seats become available again since no reservation claims them.

Movie

Movie is a lightweight entity, but it plays an important connecting role in the system. When a user searches for "Inception," the system needs to find matching Movie objects, and then use those to look up which theaters and showtimes are showing them. Movie is the glue between a user's search query and the actual bookable showtimes scattered across different theaters.

Requirement	What Movie must track
"search for movies by title"	Title string
"browse movies at a theater" / "view showtimes"	Movie identity (ID) to link showtimes

MOVIE	
<pre>class Movie: - id: string - title: string + getTitle() → string + getId() → string</pre>	

The ID exists because two movies might share a title (remakes, re-releases). The title is what users search by. If requirements expanded later (duration, genre, rating), you'd add fields to Movie and the constructor. No changes to BookingSystem, Showtime, or the booking flow.

Reservation

Reservation captures what the user booked, and provides the information needed to undo it later.

Requirement	What Reservation must track
"booking returns a confirmation ID"	Confirmation ID
"book multiple seats in a single reservation"	List of seat IDs

Requirement	What <code>Reservation</code> must track
"cancel a reservation by confirmation ID, releasing the seats"	Back-reference to the Showtime (so cancellation can be routed to the right showtime)

RESERVATION



```
class Reservation:
    - confirmationId: string
    - showtime: Showtime
    - seatIds: List<string>

    + getConfirmationId() → string
    + getSeatIds() → List<string>
    + getShowtime() → Showtime
```

The `confirmationId` is generated at booking time. This is what appears on the user's ticket and what they type in to cancel. It's the external identifier for this booking.

The back-reference to Showtime is essential for cancellation routing. When a user cancels by confirmation ID, BookingSystem needs to figure out which Showtime owns that reservation. The back-reference provides that link. Without it, you'd have to search through every theater, every showtime, and every reservation to find the right one.

Notice there's no `cancel()` method on Reservation itself. Cancellation isn't just "delete this reservation." It needs to release seats back into the available pool, which means modifying Showtime's reservation list. Reservation doesn't have access to that list and shouldn't. If Reservation could modify Showtime's state directly, we'd have two objects mutating the same data, which makes concurrency harder to reason about. So the actual cancellation logic lives on Showtime, which removes the reservation from its list and makes the seats available again.

Final Class Design

FINAL CLASS DESIGN



```
class BookingSystem:
    - theaters: List<Theater>
```

```
+ searchMovies(title: string) → List<Showtime>
+ getShowtimesAtTheater(theater: Theater) → List<Showtime>
+ book(showtimeId: string, seatIds: List<string>) → Reservation
+ cancelReservation(confirmationId: string)
```

```
class Theater:
  - id: string
  - name: string
  - showtimes: List<Showtime>

  + getShowtimes() → List<Showtime>
  + getShowtimesForMovie(movie: Movie) → List<Showtime>
```

```
class Showtime:
  - id: string
  - theater: Theater
  - datetime: DateTime
  - screenLabel: string
  - movie: Movie
  - reservations: List<Reservation>

  + getId() → string
  + getTheater() → Theater
  + getDatetime() → DateTime
  + getMovie() → Movie
  + isAvailable(seatId: string) → boolean
  + getAvailableSeats() → List<string>
  + book(reservation: Reservation)
  + cancel(reservation: Reservation)
```

```
class Movie:
  - id: string
  - title: string

  + getTitle() → string
  + getId() → string
```

```
class Reservation:
  - confirmationId: string
  - showtime: Showtime
  - seatIds: List<string>

  + getConfirmationId() → string
  + getSeatIds() → List<string>
  + getShowtime() → Showtime
```

Constants:

SEAT_LAYOUT: rows A-Z, seats 0-20 (546 seats per showtime)

Five classes, one constant. Notice the clean separation of responsibilities:

- **BookingSystem** is a pure orchestrator. It holds theaters, provides cross-theater queries (search, browse), and routes booking and cancellation requests.
- **Showtime** owns both seat state and reservations. Booking and cancellation are atomic operations on a single object, so there's no cross-object consistency to maintain.
- **Reservation** is a data record with a back-reference to its Showtime for routing cancellations. BookingSystem creates it, Showtime stores it.

Implementation



You won't implement all of this in a real interview. In practice, you'd focus on the highest-value methods — for this problem, that's `Showtime.book()` and `Showtime.cancel()`, since they're where concurrency, atomicity, and the core booking logic live. An interviewer might also ask you to walk through `BookingSystem.book()` to show the orchestration flow. The rest (search, browse, simple getters) you'd describe verbally or sketch quickly. We're covering everything here for completeness, but don't feel like you need to write all of this under time pressure.

With the class design complete, we're ready to implement the methods. We'll use pseudocode here since that's what the majority of LLD interviews want, but you'll find the complete implementation in major languages at the end as it's not uncommon for a company to have you code in the language of your choice.

For each method, we'll follow this pattern:

1. **Define the core logic** - What happens when everything goes right
2. **Handle edge cases** - Invalid inputs, concurrency, operations that can fail

The interesting work is split between two classes. Showtime handles seat state, reservation storage, and concurrency. BookingSystem orchestrates the flow, creating Reservation objects and routing them to the right Showtime. Movie and Theater are data holders. Reservation

stores booking details. We'll cover each, but expect Showtime and BookingSystem to be the longest sections.

! The concurrency requirement (R6: "exactly one succeeds" when two people book the same seat) is critical. We'll tackle this head-on when we implement `Showtime.book()`.

BookingSystem

BookingSystem is the entry point. It routes operations to the right showtime and provides the search and browse queries that cut across theaters.

The naive approach to `searchMovies` walks the entire Theater → Showtime → Movie chain every time someone searches:

BOOKINGSYSTEM.SEARCHMOVIES (NAIVE)



```
searchMovies(title)
  results = []
  searchLower = title.toLowerCase()
  now = currentTime()

  for theater in theaters
    for showtime in theater.getShowtimes()
      if showtime.getDatetime() <= now
        continue // Skip past showtimes

      movie = showtime.getMovie()
      if movie.getTitle().toLowerCase().contains(searchLower)
        results.add(showtime)

  return results
```

This works, but it's $O(t \times s)$ on every search call, where t is theaters and s is showtimes per theater. For a system where users search frequently but the movie catalog changes rarely, that repeated work is wasteful.

To improve, we can build an index at construction time. We still need the movie-to-title mapping for search, but now we're returning showtimes instead of movies, so we index both.

BOOKINGSYSTEM.CONSTRUCTOR



```
BookingSystem(theaters)
    this.theaters = theaters
    this.moviesById = {}
    this.showtimesByMovieId = {} // Map<movieId, List<Showtime>>

    for theater in theaters
        for showtime in theater.getShowtimes()
            movie = showtime.getMovie()
            moviesById[movie.getId()] = movie

            // Group showtimes by movie for efficient search
            if !showtimesByMovieId.contains(movie.getId())
                showtimesByMovieId[movie.getId()] = []
            showtimesByMovieId[movie.getId()].add(showtime)
```

Now when a user searches, we scan the movie index (a few hundred movies at most), find matches, and return their associated showtimes:

BOOKINGSYSTEM.SEARCHMOVIES



```
searchMovies(title)
    if title == null or title is empty
        return []

    results = []
    searchLower = title.toLowerCase()
    now = currentTime()

    for movie in moviesById.values()
        if movie.getTitle().toLowerCase().contains(searchLower)
            // Add all future showtimes for this movie
            for showtime in showtimesByMovieId[movie.getId()]
                if showtime.getDatetime() > now
                    results.add(showtime)

    return results
```

Same $O(t \times s)$ cost at construction, but now `searchMovies` is $O(\text{matching movies} \times \text{showtimes per movie})$, which is typically much smaller than scanning every showtime in the

system. Users get back a list of actionable showtimes showing where and when they can actually book the movie they searched for.



This constructor assumes theaters and their showtimes are fully populated at creation time. If a new showtime needs to be added later, neither BookingSystem's indexes nor Theater's list would reflect it without additional work. While unrealistic in reality, for interview scope this is usually fine, but an interviewer might ask how you'd handle dynamic additions. We explore this in the [extensibility section](#) below.

Next up, we need to be able to browse what's playing at a specific theater. Rather than returning just movie titles (which would require a second query to get showtimes), we return all showtimes at that theater. The UI can group them by movie to display "what's playing tonight."

BOOKINGSYSTEM.GETSHOWTIMESATTHEATER



```
getShowtimesAtTheater(theater)
  if theater == null
    return []

  results = []
  now = currentTime()

  for showtime in theater.getShowtimes()
    if showtime.getDatetime() > now
      results.add(showtime)

  return results
```

We only return future showtimes since past ones aren't bookable. The UI can group these by movie (using `showtime.getMovie()`) to display them like "Inception - 2pm, 5pm, 8pm" and "Dune - 3pm, 6pm, 9pm."

This approach avoids the N+1 query pattern where you'd first fetch movies at a theater, then make separate calls to get showtimes for each movie. With one call returning all showtimes, the client has everything it needs.



Won't showtimes accumulate forever? Yes, in this design old showtimes stay in memory indefinitely. Over months, the theater's showtime list grows, making this loop slower and consuming more memory. In production you'd use a database, so this isn't really a concern. But since we're working in-memory, it's worth mentioning to your interviewer that you'd want some form of cleanup for expired showtimes.

The booking flow via `book` is where BookingSystem does its orchestration work. It validates inputs, finds the right showtime, creates a Reservation, and hands it to the Showtime for atomic validation and storage.

To book, we need to find a showtime by its ID. The user picked "Inception at 7pm at AMC" from the UI, and we received a showtime ID. How do we find the corresponding Showtime object? The straightforward approach iterates all theaters and their showtimes:

```
for theater in theaters
  for showtime in theater.getShowtimes()
    if showtime.getId() == showtimeId
      // found it
```

That's $O(t \times s)$ on every booking call. We saw the exact same pattern with movies earlier, and the fix is the same: precompute a lookup map at construction time. We also need a way for cancellation to find reservations by confirmation ID. That's a similar problem, but unlike the movie and showtime maps that are built from existing data at construction, the reservation index starts empty and grows as bookings are made.

Let's update the constructor to build the `showtimesById` map (alongside the existing `moviesById` and `showtimesByMovieId`) and initialize an empty `reservationsById` index:

BOOKINGSYSTEM CONSTRUCTOR (FINAL)

```
BookingSystem(theaters)
  this.theaters = theaters
  this.moviesById = {}
  this.showtimesByMovieId = {}
  this.showtimesById = {}
  this.reservationsById = {}
```

```

for theater in theaters
    for showtime in theater.getShowtimes()
        movie = showtime.getMovie()
        moviesById[movie.getId()] = movie
        showtimesById[showtime.getId()] = showtime

        if !showtimesByMovieId.contains(movie.getId())
            showtimesByMovieId[movie.getId()] = []
            showtimesByMovieId[movie.getId()].add(showtime)

```

Three indexes built from one loop. `moviesById`, `showtimesByMovieId`, and `showtimesById` are populated at construction for efficient search and booking operations. `reservationsById` starts empty and gets populated as bookings come in, giving cancellation $O(1)$ lookup by confirmation ID.

With these indexes in place, the booking method is clean.

BOOKINGSYSTEM.BOOK



```

book(showtimeId, seatIds)
    if showtimeId == null or seatIds == null or seatIds.isEmpty()
        throw InvalidRequestException

    showtime = showtimesById[showtimeId]
    if showtime == null
        throw ShowtimeNotFoundException

    // Create the reservation up front (just a data object, no state change yet)
    reservation = Reservation(
        generateConfirmationId(),
        showtime,
        seatIds
    )

    // Hand to showtime for atomic validation + storage
    showtime.book(reservation)

    // Register in routing index so cancelReservation can find it by confirmation
    reservationsById[reservation.getConfirmationId()] = reservation

    return reservation

```

BookingSystem validates inputs, creates a Reservation with a fresh confirmation ID, then hands it to `showtime.book()`. If the showtime rejects the booking (seats unavailable), the exception propagates and the reservation is never stored. If the booking succeeds, BookingSystem registers it in the `reservationsById` routing index for future cancellation lookups.

Cancellation reverses the booking flow. The user provides a confirmation ID (from their email or ticket), and we need to find the matching reservation and route the cancel to the correct Showtime. The `reservationsById` index gives us `O(1)` lookup, and the reservation's back-reference tells us which Showtime to route to.

BOOKINGSYSTEM.CANCELRESERVATION



```
cancelReservation(confirmationId)
    if confirmationId == null or confirmationId is empty
        throw InvalidRequestException

    reservation = reservationsById[confirmationId]
    if reservation == null
        throw ReservationNotFoundException

    // Follow the back-reference to the correct showtime
    showtime = reservation.getShowtime()

    // Showtime removes the reservation and frees the seats atomically
    showtime.cancel(reservation)

    // Remove from routing index
    reservationsById.remove(confirmationId)
```

The flow follows a similar pattern as booking. Look up the reservation, follow the back-reference, and tell the Showtime to cancel it. All the actual state changes (removing the reservation, freeing the seats) happen inside `showtime.cancel()` under a synchronized block. BookingSystem just cleans up its own routing index afterward.

Theater

Theater holds a list of showtimes and provides methods to query them. The constructor initializes an empty showtime list — showtimes are added after construction since each

Showtime needs a reference back to its Theater. We expose `getShowtimes()` to return the full list and `getShowtimesForMovie()` to filter by a specific movie.

THEATER



```
class Theater:
    id: string
    name: string
    showtimes: List<Showtime>

    Theater(id, name)
        this.id = id
        this.name = name
        this.showtimes = []

    getShowtimes()
        return showtimes

    getShowtimesForMovie(movie)
        results = []
        for showtime in showtimes
            if showtime.getMovie().getId() == movie.getId()
                results.add(showtime)
        return results
```

`getShowtimesForMovie` is a convenience filter that iterates showtimes and returns those matching the given movie. Theater knows nothing about booking logic. It doesn't validate seats or track reservations. That separation matters because if booking rules change, we only modify Showtime, not Theater.

Showtime

Everything we've built so far feeds into this class. BookingSystem routes requests. Theater and Movie hold data. But Showtime is where state actually changes. Every booking adds a reservation here. Every cancellation removes one. Every availability check reads from here. If you get Showtime right, the rest of the system is just plumbing.

This is also the class interviewers care most about, because it touches all the hard parts at once: state management, concurrency, atomicity, and validation. We'll work through each of those as we implement the methods.

The constructor sets up the identity fields like its ID, what movie it's showing, the datetime, and which screen it's on. The interesting part is the `reservations` list. It starts empty, and every booking appends to it. Every cancellation removes from it. This single list tracks all seat state for this showtime — no separate "bookedSeats" tracker, no availability flags, no secondary data structure. If you want to know whether seat A5 is taken, you ask the reservations.

SHOWTIME.CONSTRUCTOR



```
Showtime(id, theater, movie, datetime, screenLabel)
  this.id = id
  this.theater = theater
  this.movie = movie
  this.datetime = datetime
  this.screenLabel = screenLabel
  this.reservations = []
```

So how do we actually answer "is seat A5 taken?" The implementation follows directly from the reservations list. Walk through each reservation, check if any of them claim the seat. If none do, it's available.

SHOWTIME.ISAVAILABLE



```
isAvailable(seatId)
  for reservation in reservations
    if reservation.getSeatIds().contains(seatId)
      return false
  return true
```

Nothing but a linear scan through the reservations. For a typical showtime with a few dozen reservations, this is fast enough that there's no reason to optimize it further.

When a user opens the seat picker, they need to see every seat in the theater, with booked ones grayed out. `getAvailableSeats` produces that full picture. The approach is to first figure out what's taken by collecting all booked seats into a set, then walk through the constant layout and keep anything that isn't in the set.

SHOWTIME.GETAVAILABLESEATS



```
getAvailableSeats()
    booked = Set()
    for reservation in reservations
        for seat in reservation.getSeatIds()
            booked.add(seat)

    available = []
    for row in 'A' to 'Z'
        for num in 0 to 20
            seatId = row + num
            if !booked.contains(seatId)
                available.add(seatId)
    return available
```

Building the booked set is $O(\text{total booked seats})$ and the layout scan is a fixed $O(546)$. A frontend seat picker calls this once when the user opens the page, so the cost is negligible.

You could also maintain a persistent `bookedSeats` Set on Showtime for $O(1)$ availability checks instead of scanning reservations. That's a perfectly valid approach. The tradeoff is that you'd have a second mutable field that needs to stay in sync with `reservations` on every book and cancel call. Either way works fine at this scale. We'll go with deriving availability from the reservations list to keep the mutable state minimal, but a `bookedSeats` set is equally defensible.

Everything up to this point has been read operations, checking availability, listing seats. Those are safe on their own since they don't modify anything, but they can still see inconsistent state if another thread is mid-write. But `book()` is different. It adds a reservation to the list, which means seats that were available a moment ago are suddenly taken. If two threads both check seat A5, both see it's available, and both add their reservation, we've double-booked the seat. Two customers show up to the same chair. Our requirement (R6) says exactly one booking should succeed, so we need to make the check-and-store sequence atomic.

Bad Solution: Check then book (no synchronization)



Great Solution: Synchronized booking with atomic check-and-set



There's a third option worth considering. With per-showtime locking, if Alice is booking seats A5 and A6 for the 7pm Inception showing, Bob has to wait even if he wants completely different seats like M12 and M13 for the same showing. Those operations don't actually conflict with each other, but the lock doesn't know that. For a regular Tuesday afternoon showing, this doesn't matter. But imagine opening night of a Marvel movie, with hundreds of people all trying to book seats at the same time. Every single one of them queues up behind the same lock, even though they're all after different seats.

The alternative is to lock at the seat level instead of the showtime level. Each seat gets its own lock, so Alice grabbing A5 and A6 only blocks someone else who wants those exact seats. Bob's booking of M12 and M13 proceeds in parallel without waiting.

Good Solution: Fine-grained locking (per-seat)



Per-seat locking is a legitimate choice and there's no wrong answer here. But for this design, per-showtime locking hits the sweet spot in my opinion. The critical section is short (a scan through reservations and a list append), so even when two users want the same showtime, one waits briefly while the other finishes. Seat stays a simple string, there's no deadlock risk, and there's only one piece of mutable state to reason about. If an interviewer pushes on throughput for a massively popular showtime, per-seat locking is the natural next step, but it's an optimization for when the simpler approach becomes a measured bottleneck.

We'll go with synchronized booking. Here's the complete implementation:

SHOWTIME.BOOK



```
book(reservation)
    synchronized(this)
        seatIds = reservation.getSeatIds()

        if seatIds == null or seatIds.isEmpty()
            throw InvalidRequestException("Must select at least one seat")

        // Validate all seats exist in the layout
        for seatId in seatIds
            if !isValidSeatId(seatId)
                throw InvalidSeatException(seatId)

        // Check all seats are available
```

```
for seatId in seatIds
    if !isAvailable(seatId)
        throw SeatUnavailableException(seatId)

// All checks passed - store reservation
reservations.add(reservation)

isValidSeatId(seatId)
    row = seatId[0]
    num = parseInt(seatId.substring(1))
    return row >= 'A' && row <= 'Z' && num >= 0 && num <= 20
```

The method validates seat IDs, checks availability, and stores the reservation — all inside a single synchronized block so the check and state change are atomic. If any seat is unavailable, the booking fails with no state change.

Now the cancel method:

SHOWTIME.CANCEL



```
cancel(reservation)
    synchronized(this)
        reservations.remove(reservation)
```

Remove the reservation from the list. That's it. The seats become available automatically because `isAvailable` derives its answer from the reservation list. No secondary data structure to clean up.

We synchronize cancel for the same reason we synchronize book. Without the lock, a concurrent booking could scan reservations in the middle of our removal, seeing a partially modified list. The synchronized block ensures the removal is atomic with respect to any concurrent booking attempts.

Movie

Movie needs just two pieces of data based on current requirements, an ID and a title. Why both? Because titles aren't unique. "Dune" from 2021 and "Dune" from 1984 are different movies that a theater might both show during a retrospective. The ID (a UUID or database key) uniquely identifies each movie, while the title is what users type into the search box.

The constructor takes both values and stores them as private fields. We expose read-only getters (`getId()` , `getTitle()`) but no setters. Once a movie is created, its data doesn't change within our system. This immutability makes the class easier to reason about, especially when the same Movie object is referenced by multiple Showtime instances.

MOVIE



```
class Movie:
    id: string
    title: string

    Movie(id, title)
        this.id = id
        this.title = title

    getId()
        return id

    getTitle()
        return title
```

Reservation

Reservation stores three things: the confirmation ID (what the user sees on their ticket), a back-reference to the Showtime (for routing cancellations), and the list of seat IDs that were booked. The constructor takes all three, and we expose them through getters. Notice the defensive copies on `seatIds` :

RESERVATION



```
class Reservation:
    confirmationId: string
    showtime: Showtime
    seatIds: List<string> // e.g., ["A5", "A6"]

    Reservation(confirmationId, showtime, seatIds)
        this.confirmationId = confirmationId
        this.showtime = showtime
        this.seatIds = copy(seatIds) // Defensive copy

    getConfirmationId()
```

```

        return confirmationId

    getShowtime()
        return showtime

    getSeatIds()
        return copy(seatIds) // Return copy to prevent modification

```

Complete Code Implementation

While most interviews only require pseudocode, some ask for working code. Below is a complete implementation in common languages for reference.

<
 BookingSystem.py
 Showtime.py
 Theater.py
 Movie.py
>
Python
▼


```

import uuid
from datetime import datetime

class BookingSystem:
    def __init__(self, theaters: list["Theater"]):
        self._theaters = theaters
        self._movies_by_id: dict[str, "Movie"] = {}
        self._showtimes_by_movie_id: dict[str, list["Showtime"]] = {}
        self._showtimes_by_id: dict[str, "Showtime"] = {}
        self._reservations_by_id: dict[str, "Reservation"] = {}

        for theater in theaters:
            for showtime in theater.get_showtimes():
                movie = showtime.get_movie()
                self._movies_by_id[movie.get_id()] = movie
                self._showtimes_by_id[showtime.get_id()] = showtime

                if movie.get_id() not in self._showtimes_by_movie_id:
                    self._showtimes_by_movie_id[movie.get_id()] = []
                self._showtimes_by_movie_id[movie.get_id()].append(showtime)

    def search_movies(self, title: str) -> list["Showtime"]:
        if not title:
            return []

        results: list["Showtime"] = []
        search_lower = title.lower()

```

Verification

Let's trace through scenarios to verify our implementation handles the key cases correctly.

Test Case 1: Successful Booking Flow

Setup: Showtime for "Inception" at 7pm with all 546 seats available.

Initial state:

```
showtime.reservations = [] (empty)
bookingSystem.reservationsById = {} (empty)
```

Operation: `bookingSystem.book("showtime-123", ["A5", "A6"])`

`BookingSystem.book`:

1. Validate inputs → OK
2. Look up showtime: `showtimesById["showtime-123"]` → found
3. Create reservation:
`confirmationId = generateConfirmationId() → "BMS-X7Y2Z9K4"`
`showtime = showtime`
`seatIds = ["A5", "A6"]`
4. `showtime.book(reservation)`:
`synchronized(this)`:
 Validate "A5": `isValidSeatId` → true ✓
 Validate "A6": `isValidSeatId` → true ✓
 Check "A5": `isAvailable("A5")` → true ✓
 Check "A6": `isAvailable("A6")` → true ✓
 Store: `reservations.add(reservation)`
 (returns without throwing)
5. Register in routing index: `reservationsById["BMS-X7Y2Z9K4"] = reservation`
6. Return reservation

Final state:

```
showtime.reservations = [reservation] (contains seats "A5", "A6")
bookingSystem.reservationsById = {"BMS-X7Y2Z9K4" → reservation}
```

Result: Reservation with confirmation "BMS-X7Y2Z9K4" ✓

The booking succeeded. BookingSystem created the Reservation and handed it to Showtime, which validated seats and stored it atomically. BookingSystem then registered it in the routing index for future cancellation lookups.

Test Case 2: Concurrent Booking - Exactly One Succeeds

Setup: Showtime with seat "A5" available. Two threads try to book it simultaneously.

Initial state:

```
showtime.reservations = [] (empty)
```

Thread A: bookingSystem.book("showtime-123", ["A5"])

Thread B: bookingSystem.book("showtime-123", ["A5"])

Both threads create their Reservation objects, then call showtime.book():

Thread A: enters synchronized block

Thread B: waits at synchronized block entrance

Thread A (inside lock):

Check "A5": isAvailable("A5") scans reservations → true ✓

Store: reservations.add(reservation)

Exit synchronized block

Thread A (back in BookingSystem): registers in routing index

Thread B (now enters lock):

Check "A5": isAvailable("A5") scans reservations → false × (Thread A's reservation)

Throw SeatUnavailableException("A5")

Thread B (exception propagates to BookingSystem): reservation never registered

Result:

Thread A: Returns reservation ✓

Thread B: Throws SeatUnavailableException ✓

Exactly one thread succeeded. Requirement R6 satisfied.

The synchronized block ensures only one thread can check-and-store at a time. Thread B's reservation was created but never persisted because the showtime rejected it.

Test Case 3: Cancellation Releases Seats Correctly

Setup: Existing reservation "BMS-X7Y2Z9K4" for seats "A5" and "A6".

Initial state:

```
showtime.reservations = [reservation] (reservation holds seats "A5", "A6")
bookingSystem.reservationsById = {"BMS-X7Y2Z9K4" → reservation}
reservation.showtime = showtime
```

Operation: `bookingSystem.cancelReservation("BMS-X7Y2Z9K4")`

BookingSystem.cancelReservation:

1. Look up: `reservationsById["BMS-X7Y2Z9K4"]` → found
2. Follow back-reference: `reservation.getShowtime()` → showtime
3. `showtime.cancel(reservation):`
 `synchronized(this):`
 `reservations.remove(reservation)`
4. Remove from routing index: `reservationsById.remove("BMS-X7Y2Z9K4")`

Final state:

```
showtime.reservations = [] (empty, seats "A5" and "A6" now available)
bookingSystem.reservationsById = {} (empty)
```

Result: Seats "A5" and "A6" are now available for new bookings ✓

Showtime removed the reservation, making the seats available again. BookingSystem cleaned up its routing index.

Test Case 4: Partial Booking Fails Atomically

Setup: Seat "A6" is already booked. User tries to book "A5", "A6", and "A7".

Initial state:

```
showtime.reservations = [existing_reservation] (existing_reservation holds seat
```

Operation: `bookingSystem.book("showtime-123", ["A5", "A6", "A7"])`

BookingSystem.book:

1. Look up showtime → found

```
2. Create reservation with seatIds = ["A5", "A6", "A7"]
```

```
3. showtime.book(reservation):
```

```
    synchronized(this):
```

```
        Validate "A5", "A6", "A7": all valid seat IDs ✓
```

```
        Check "A5": isAvailable("A5") → true ✓
```

```
        Check "A6": isAvailable("A6") → false ✗ (claimed by existing_reservation)
```

```
        Throw SeatUnavailableException("A6")
```

```
        // Never reaches the store step!
```

```
4. Exception propagates to BookingSystem
```

```
5. Reservation never registered in routing index
```

```
Final state:
```

```
    showtime.reservations = [existing_reservation] (unchanged!)
```

```
Result:
```

```
    Exception thrown ✓
```

```
    "A5" was NOT booked (all-or-nothing) ✓
```

```
    No reservation stored anywhere ✓
```

No partial booking occurred — seat A5 was never claimed despite being available, because A6 failed the check first. The reservation was created but never persisted.

Extensibility

If there's time after implementation, interviewers like to probe whether your design can evolve without a rewrite. You typically won't implement these changes during the interview. Instead, you'll explain where they'd fit and what ripple effects they'd have.

How would you support dynamically adding and removing showtimes, movies, and theaters?

The current design assumes everything is configured at construction time. The constructor scans all theaters and showtimes in one pass, builds the lookup indexes, and never touches them again. That's a fine simplification for interview scope, but any interviewer who read the constructor will notice it. In reality, theaters add new showtimes constantly. Next week's schedule, a surprise late-night showing, a new movie premiere.

Adding is the easy direction. Expose an `addShowtime` method on `BookingSystem` that updates every data structure the constructor populated.

BOOKINGSYSTEM.ADDSHOWTIME



```
addShowtime(theater, showtime)
  theater.getShowtimes().add(showtime)
  showtimesById[showtime.getId()] = showtime

  movie = showtime.getMovie()
  moviesById[movie.getId()] = movie

  if !showtimesByMovieId.contains(movie.getId())
    showtimesByMovieId[movie.getId()] = []
  showtimesByMovieId[movie.getId()].add(showtime)
```

Each line corresponds to one of the indexes the constructor built. The theater's showtime list gets the new entry so `getShowtimes` and `getShowtimesForMovie` return it. The `showtimesById` index gets updated so `book()` can route to this showtime by ID. The `moviesById` and `showtimesByMovieId` indexes get updated so `searchMovies` will find and return this showtime for matching movie titles. The `idempotent` map writes handle both new and existing movies naturally.

The downside is that every index the constructor populates needs a corresponding update in `addShowtime`. If you later add a new index (say, a map from movie ID to its list of showtimes for faster filtering), you need to remember to update it here too. This is the classic tradeoff with denormalized data. Reads get faster, but writes get more error-prone because you have multiple data structures that must stay in sync.

For thread safety, `addShowtime` needs synchronization if showtimes can be added while bookings are in progress. The simplest approach is a write lock on `BookingSystem` for structural changes, while the existing per-showtime `synchronized` blocks continue handling booking concurrency independently. A new showtime has no reservations yet, so there's no contention with the booking path.

Removing is harder, because entities in this system reference each other and pulling one out can leave dangling references. If someone has tickets for the 7pm Inception showing and you delete that showtime, their reservation points at nothing. Their confirmation ID leads nowhere.

The cleanest approach for interview scope is to reject removal if active reservations exist. If a theater needs to cancel a showing, they should cancel all existing reservations first (notifying customers through whatever channel), and only then remove the showtime from the system. That fan-out is the **Observer pattern**: one event, several listeners that each decide how to reach the customer (email, SMS, push) without Showtime knowing about any of them. It's the same shape the **inventory management breakdown** uses for its low-stock alerts.

BOOKINGSYSTEM.REMOVESHOWTIME



```
removeShowtime(showtimeId)
    showtime = showtimesById[showtimeId]
    if showtime == null
        throw ShowtimeNotFoundException

    if showtime.getReservations() is not empty
        throw ShowtimeHasActiveReservationsException

    showtimesById.remove(showtimeId)

    // Remove from movie → showtimes index
    movie = showtime.getMovie()
    if showtimesByMovieId.contains(movie.getId())
        showtimesByMovieId[movie.getId()].remove(showtime)

    for theater in theaters
        theater.getShowtimes().remove(showtime)

    cleanupMovieIndex(movie)
```

The movie cleanup at the end deserves attention too. A movie only exists in the system because some showtime references it. If you remove the last showtime for "Inception," should "Inception" disappear from search results? Yes, there's nothing bookable. But you need to verify no other showtime still references this movie before pulling it from the index.

BOOKINGSYSTEM.CLEANUPMOVIEINDEX



```
cleanupMovieIndex(movie)
    for theater in theaters
        for showtime in theater.getShowtimes()
            if showtime.getMovie().getId() == movie.getId()
                return // Still showing somewhere, keep it
```



```
moviesById.remove(movie.getId())  
showtimesByMovieId.remove(movie.getId())
```

This is $O(t \times s)$ in the worst case, but it only runs on showtime removal, which we expect to be an exceptional scenario.

Adding a theater follows the same pattern as adding a showtime. Create the Theater object, add it to the theaters list, and index any showtimes it comes with. Removing a theater is the hardest operation because it cascades. You need to handle (or reject based on) all active reservations across all of that theater's showtimes, remove every showtime from `showtimesById`, and clean up movies that are no longer showing anywhere.

In production, you'd just throw an exception if there are active tickets. If a theater is closing, you'd stop adding new showtimes and let the existing ones expire naturally.

How would you handle temporary seat holds during checkout?

Right now, booking is instantaneous. The user picks seats and `book()` either succeeds or fails. But in a real system, there's a gap between "user selects seats" and "payment completes." The user opens the seat picker, chooses A5 and A6, then spends 30-60 seconds entering their credit card. During that window, another user could grab those same seats. The first user fills out the entire payment form only to get "seats no longer available" at the end. That's a bad experience.

The fix is to introduce a temporary hold. When a user selects seats, we reserve them for a limited time (say, 5 minutes). During that window, those seats appear as unavailable to everyone else. If the user completes payment within the window, the hold converts into a confirmed reservation. If they abandon checkout or the timer expires, the seats release back into the pool.

This requires changing how Showtime tracks seat state. Instead of a binary "available or booked," seats now have three states: available, held, and booked. That's a small lifecycle, and once the transitions carry real rules (you can't hold a booked seat, you can't book a seat someone else is holding), it's exactly the kind of thing the State pattern exists for. For three states it'd be overkill, but if an interviewer keeps piling on states (reserved, in-payment, refunded), modeling each as its own object with explicit legal transitions is the move.

SHOWTIME WITH SEAT HOLDS



```
class Showtime:
    - id: string
    - theater: Theater
    - datetime: DateTime
    - screenLabel: string
    - movie: Movie
    - reservations: List<Reservation>
    - holds: Map<string, SeatHold>

class SeatHold:
    - seatIds: List<string>
    - holdId: string
    - expiresAt: long
```

The `holds` map tracks active seat holds by hold ID. Each `SeatHold` records which seats are held and when the hold expires. The `reservations` list continues to track confirmed bookings, same as before.

This changes `isAvailable`. Previously, a seat was available if no reservation claimed it. Now, a seat is unavailable if it's either booked by a confirmed reservation or held by a non-expired hold. We check both.

SHOWTIME.ISAVAILABLE (WITH HOLDS)



```
isAvailable(seatId)
    for reservation in reservations
        if reservation.getSeatIds().contains(seatId)
            return false

    now = currentTime()
    for hold in holds.values()
        if hold.expiresAt > now && hold.seatIds.contains(seatId)
            return false

    return true
```

With availability updated, the booking flow splits into two steps instead of one. The original `book()` method did everything atomically: check seats, store reservation, done. Now there's a time gap in the middle where the user is completing payment, so we need to split the

operation. First, when the user selects seats on the seat picker and clicks "proceed to checkout," we place a hold.

SHOWTIME.HOLDSEATS



```
holdSeats(seatIds, timeoutMs)
    synchronized(this)
        for seatId in seatIds
            if !isAvailable(seatId)
                throw SeatUnavailableException(seatId)

        hold = SeatHold(
            seatIds,
            generateHoldId(),
            currentTime() + timeoutMs
        )
        holds[hold.holdId] = hold
        return hold.holdId
```

The user now has 5 minutes (or whatever timeout we configured) to complete payment. Their seats are protected during this window. Other users opening the seat picker will see those seats as unavailable.

When payment succeeds, the hold converts into a confirmed reservation. The `confirmHold` method validates that the hold still exists and hasn't expired, removes it from the holds map, and adds the reservation to the list. If the hold expired between payment submission and confirmation (a tight race but possible), we reject it and the user has to start over.

SHOWTIME.CONFIRMHOLD



```
confirmHold(holdId, reservation)
    synchronized(this)
        hold = holds[holdId]
        if hold == null
            throw HoldNotFoundException

        if currentTime() > hold.expiresAt
            holds.remove(holdId)
            throw HoldExpiredException

        // Hold is valid, convert to reservation
```

```
holds.remove(holdId)
reservations.add(reservation)
```

What about users who abandon checkout? They select seats, get a hold, then close their browser or get distracted. The hold sits in memory with no one coming back to confirm or cancel it. Those seats stay locked until the expiration time passes, but expired holds don't clean themselves up. We need a background cleanup task that periodically scans for expired holds and removes them.

SHOWTIME.CLEANUPEXPIREDHOLDS



```
cleanupExpiredHolds()
    synchronized(this)
        now = currentTime()
        for holdId in holds.keys()
            if now > holds[holdId].expiresAt
                holds.remove(holdId)
```

Once a hold is removed, those seats pass the `isAvailable` check again and return to the pool.



The timeout value is a business decision. Too short (30 seconds) and you cancel holds while users are still typing their credit card number. Too long (15 minutes) and you lock up seats for people who've already moved on. Most ticketing systems use 5-10 minutes. Some get clever with adaptive timeouts, giving shorter holds during high-demand events like opening night of a Marvel movie and longer holds during off-peak times. If you wanted to support a few of these policies and switch between them, you'd pull the timeout calculation behind an interface and let Showtime hold whichever one it was given. That's the [Strategy pattern](#), the same move the [parking lot breakdown](#) uses for pricing rules.

This also fits cleanly into the existing concurrency model. Both `holdSeats` and `confirmHold` run inside `synchronized(this)`, same as `book` and `cancel`. That means holds and bookings serialize against each other on the same showtime lock. A hold can't sneak in between another thread's availability check and reservation storage, which is exactly the guarantee we need. The per-showtime lock we chose earlier scales naturally to support this feature without any changes to the locking strategy.

What is Expected at Each Level?

This one was a little more complex than some of the other breakdowns, so what is actually expected at each level?

Junior

At the junior level, I'm looking for whether you can identify the core entities and build a working booking flow. You should recognize that BookingSystem orchestrates things, Showtime tracks seat state, and Reservation captures what was booked. Your `book` method should check seat availability and store a reservation, and `cancel` should remove it and free the seats. Basic validation matters: reject invalid seat IDs, handle the case where a seat is already taken. You might not see the concurrency issue on your own, and that's fine at this level. If I ask "what happens if two users try to book the same seat at the same time?", needing a hint toward synchronization is expected. The key is demonstrating you can model the entities, wire them together, and handle the straightforward booking and cancellation paths.

Mid-level

Mid-level candidates should produce a cleaner separation between BookingSystem as the orchestrator and Showtime as the owner of seat state and reservations. You should recognize that the reservations list on Showtime is the single source of truth for which seats are booked, and that availability can be derived from it without a separate data structure. I expect awareness of the concurrency problem and a working solution using synchronized booking with the check-and-store sequence wrapped in a single lock. The all-or-nothing behavior for multi-seat bookings should be handled correctly: if one seat in a group is taken, the entire booking fails with no partial state change. You should also use lookup indexes (like `showtimesById` and `reservationsById`) for efficient routing rather than scanning all theaters on every operation, though you might build these incrementally rather than planning them upfront.

Senior

Senior candidates should nail the concurrency model without prompting. You should proactively explain the check-then-act race condition in booking and why the availability check

and reservation storage must be atomic. I expect you to discuss the tradeoff between per-showtime locking (simpler, no deadlock risk, good enough for most cases) and per-seat locking (higher throughput, but requires sorted lock acquisition to avoid deadlock and promotes Seat from a string to a class). The back-reference from Reservation to Showtime for cancellation routing should come naturally as part of your design, not as an afterthought. You should also be comfortable discussing extensibility like temporary seat holds during checkout, explaining how `holdSeats` and `confirmHold` fit into the existing synchronization model without changing the locking strategy.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 Start Quiz